

TransLib: An Extensible Graph-Aware Library Framework for Automated Generation of Transformer Operators on FPGA

Yang Liu*, Tianchen Wang, Yuxuan Dong, Zexu Zhang, Shun Li, Jun Yu, Kun Wang†

School of Microelectronics, Fudan University, Shanghai, China

*yang_liu22@m.fudan.edu.cn, †kun.wang@ieee.org

ABSTRACT

Advanced transformer algorithms are emerging in abundance, finding widespread applications across numerous fields. However, as a crucial deployment strategy, the design of FPGA-based transformer accelerators is progressing relatively slowly and laboriously. In this paper, we propose *TransLib*, an automated and extensible library-based framework for the operator sets generation of transformer networks, to enable agile design of FPGA-based accelerators. Specifically, *TransLib* first accomplishes expertise engagement with graph awareness via novel stepwise-contraction analysis and keypoint-centric matching in the frontend. Subsequently, the backend carries out design space mining and operator generation for various platforms and scenarios. Additionally, as the basis of these two parts, a pluggable template library is built with succinct and bidirectional interfaces, allowing *TransLib* to be extensible and customizable. Experimental results show that (1) *TransLib* is capable of engaging expertise in the generic design flow for typical transformer networks; (2) *TransLib* can flexibly support over 10^{10} to 10^{12} different types of operator generation for a given network, each with varying tradeoffs in resource and performance; (3) *TransLib* fairly leverages resources of target platform from edge to data center, while achieving considerable speedup compared to CPU and competitive performance compared to previous expert-crafted accelerators.

ACM Reference Format:

Yang Liu*, Tianchen Wang, Yuxuan Dong, Zexu Zhang, Shun Li, Jun Yu, Kun Wang†. 2024. TransLib: An Extensible Graph-Aware Library Framework for Automated Generation of Transformer Operators on FPGA. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '24)*, October 27–31, 2024, New York, NY, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3676536.3676749>

1 INTRODUCTION

Transformer networks [7, 8, 22] have achieved great success in a variety of deep learning tasks, such as natural language processing (NLP) and computer vision (CV). Inference of transformer networks is inefficient on general-purpose architectures, making accelerators attractive. Several FPGA-based or ASIC-based transformer accelerators are proposed [2, 3, 13, 17]. Although they enable acceleration

of the transformer network on hardware platform, excessive manual optimization limits their generality and automation. Moreover, As multiple variants of transformer networks evolve, there is an increasing demand for automated design tools that possess both robustness and flexibility.

Accelerator design automation has gained significant popularity across various applications. Generation tools for fine-grained operators [12, 18, 26] are able to achieve high performance operators automatically through precise algorithms and specific optimization. High-level synthesis (HLS) tools with pure compilation [1, 4, 5, 25] have been developed to enable easy implementation of hardware acceleration for numerous general purpose algorithms or simple neural networks (NNs). Furthermore, automated generation frameworks of deep neural networks (DNNs) [21, 24] utilize specifications and self-developed libraries to realize automation and agile design of DNN accelerators, bridging the gap between automation tools and expert-crafted design.

However, these works mentioned above are not directly applicable to the automatic generation of transformer network accelerators. First, generation tools for fine-grained operators focus on individual kinds of fine-grained operators, which cannot support the entire accelerator design of transformer networks. Second, immense and complex transformer network structure brings an unbearable workload on compiler of HLS tools. And the absence of expertise involvement remains a hindrance to efficient generation, rendering completely automated compilation unfeasible. Finally, the dramatic difference between DNN and transformer on both the dataflow and network structures hinders the use of previous frameworks directly. In addition, although some of these tools achieve a certain level of expertise engagement, their usability for users is significantly limited due to their lack of extensibility.

Motivated by the issues above, we propose *TransLib*, an extensible library-based framework for automated generation of transformer operator sets on FPGA. It first reduces model complexity through frontend graph structure awareness, then collaborates with a template library to achieve expertise engagement, and finally explores the design space in the backend to obtain a hardware operator set of the transformer network suiting the target platform. To the best of our knowledge, *TransLib* is the first automated and extensible generation framework capable of transformer-type networks. Table 1 summarizes the comparison of features between the most relevant state-of-the-art work and *TransLib*.

In particular, the main contributions of this paper are as follows:

- **TransLib.** We propose an extensible library-based framework called *TransLib*, which enables automated and agile design of transformer accelerators. Specifically, the frontend conducts graph analysis and optimization, then the backend completes the design space exploration and operator generation on top of the template library.

†Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICCAD '24, October 27–31, 2024, New York, NY, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1077-3/24/10...\$15.00
<https://doi.org/10.1145/3676536.3676749>

- **Graph Awareness.** The frontend of *TransLib* features innovative graph analysis and matching algorithms, ideal for large-scale networks, to achieve expertise engagement by structure awareness.
- **Flexible Generation.** The backend of *TransLib* exploits wide design space through reuse and dataflow scheduling, allowing for flexible generation for different target platforms and scenarios.
- **Extensible Library.** The template library boasts an extensible design with concise and bidirectional interfaces, enabling hardware developers to plug in customizable template with ease.

The rest of this paper is organized as follows. Section 2 introduces prior studies on transformer and operator generation. Section 3 provides an overview of *TransLib*. Section 4 presents details of the framework methodology. Section 5 illustrates the evaluation results of *TransLib*, and Section 6 gives the conclusion.

2 PRELIMINARIES

Transformer Fundamental. Transformer is an NN architecture originally proposed in 2017 [22]. Typical sequential models suffer from issues like vanishing and exploding gradients when processing long sequences. Transformer network addresses these problems by introducing a self-attention mechanism, enabling parallel computation of representations for each position in the sequence and capturing long-range dependencies more effectively. On this basis, multiple variants of transformer networks evolve. BERT [7] and GPT [9] excel at NLP and have become milestones in pretraining techniques. Moreover, ViT [8] applies the transformer concept on the field of CV, which breaks the reliance on convolutional layers in traditional vision model and performs excellently in many vision tasks through fine-tuning or transfer learning. Meanwhile, a few corresponding hardware accelerators have emerged [13, 17].

Compilation-based HLS Tools. HLS tools are used to generate HDL code from high-level language code for hardware design. Typical compiler toolchains like TVM [4] are degraded and optimized based on LLVM [15] representations to automate the mapping of algorithms. In recent years, MLIR infrastructures [16] are widely used to support compilation-based tools to achieve cascading degradation of HLS [1, 5, 25]. Nevertheless, transformer networks consist of repeated computation blocks with multiple arithmetic units and complex dataflow, making existing HLS tools incapable of optimizing transformer networks, and for most of them, it is either necessary to utilize commercial tools as the toolchain backend or difficult to employ expertise techniques in hardware design. Furthermore, some innovative HLS tools like TRAC [20] combine

HLS-based libraries to support transformer accelerator design, yet these tools are lacking in sufficient design space exploration (DSE), making the design inflexible and inextensible.

Generation Framework. The great demand for developing FPGA/ASIC-based accelerators of fast-growing neural networks propels the development of automated accelerator generation frameworks. For example, FINN [21] is a framework for building fast and flexible FPGA accelerators for binarized neural networks. AutoDNNchip [24] is able to automatically generate both FPGA- and ASIC- based DNN accelerators with chip predictor and builder. In addition to these frameworks, generation tools for fine-grained operators are often used to improve performance of the accelerator. NN-LUT [26] is a low-cost and accurate generation tool for non-linear operators in transformer models via combination of LUT approximation and neural networks. Besides, TensorLib [12] optimizes operator implementation through specific scheduling of tensor dataflows. However, the significant complexity in dataflows and network structures of transformer-type networks prevent the direct application of existing frameworks.

3 OVERVIEW OF TRANSLIB

Fig. 1 provides an overview of *TransLib*, which can automatically generate optimized hardware operator sets for transformer networks. *TransLib* takes the ONNX [6] modelproto of networks as the input, automatically generating the optimized hardware description language (HDL) code on top of its pluggable template library. For different target platforms, such as data centers and resource-constrained edge platforms, *TransLib* is capable of outputting various operator sets that serve as the foundation for the assembly of subsequent accelerators. The realization of *TransLib* is achieved by the collaboration among the proposed frontend with the parser and optimizer; backend with DSminer and builder; and template library with configurable operator templates and generation methods.

To be specific, the four-step generation flow of *TransLib* can be presented as follows. First, for the input modelproto of transformer networks, the parser extracts the necessary network information and fixes a few possible problems caused by ONNX framework (①). Afterwards, the optimizer constructs the computational flow with parsed network and performs expertise engaged optimizations with structure awareness via stepwise-contraction analysis and keypoint-centric matching (② in Section 4.1). At the end of frontend processing, the DSminer completes the design space mining from three perspectives (③ in Section 4.2). Then, based on internal information of operators, the builder searches in the template library and automatically generates corresponding hardware modules with standard interfaces of I/O ports (④ in Section 4.4).

One of the major challenges *TransLib* needs to overcome is the collaborative scheduling between the library and the rest of *TransLib* toolchain, as this is the key to involving expertise in the optimization of the operator design. To address this challenge, we adopt a clear and bidirectional configuration flow around the template library (in Section 4.3). On the one side, the library provides a configure set of operator topologies, which can be passed to the frontend as a basis for expertise-involved optimization. On the other side, a description file presents a hierarchical description of

Table 1: Features comparison among relevant work.

	Input	Output	F1	F2	F3	F4	F5
TVM [4]	ProtoBuf	LLVM IR	✗	✗	✗	✗	✓
SODA-OPT [1]	MLIR	LLVM IR	✗	✗	✗	✗	✓
ScaleHLS [25]	MLIR	LLVM IR, C++	✗	✗	✗	✗	✓
CIRCT HLS [5]	MLIR	HDL	✗	✓	✗	✗	✗
FINN [21]	----	Bitstream	✗	✓	✓	✗	✓
AutoDNNchip [24]	User-Defined	HDL, C++	✗	✓	✓	✗	✓
TRAC [20]	----	C++	✓	✗	✓	✗	✓
TransLib	ProtoBuf	HDL	✓	✓	✓	✓	✓

F1: Transformer Support; F2: No Additional Backend Needed;
F3: Expertise Engagement; F4: Extensible Library; F5: DSE

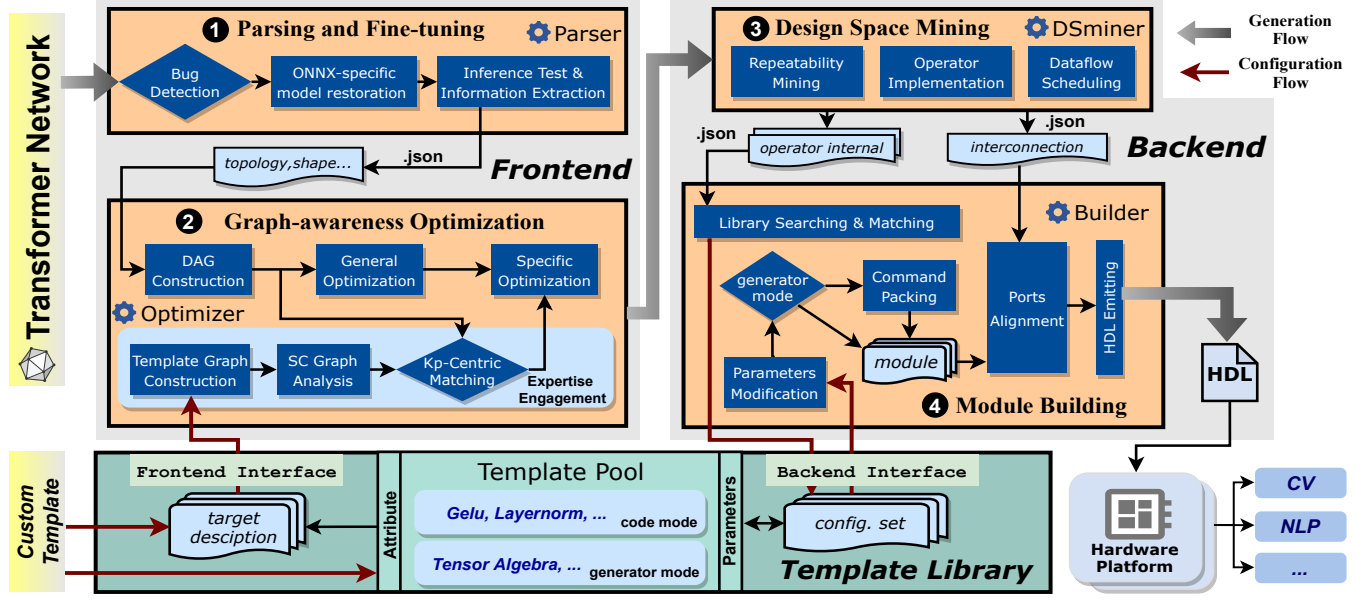


Figure 1: The overall framework of *TransLib*. (a) Frontend: Extract network information and attain network optimization through expertise engagement. (b) Backend: Perform design space mining and generate hardware modules of operator sets. (c) Template library: Store hardware templates and interact with the frontend and backend.

operator generation methods contained in the library, which can be utilized by the backend for module generation. Moreover, due to its concise interfaces, customizable configuration is also achieved in *TransLib* for extensible library and optional design space setup.

4 METHODOLOGY

This section focuses on some specific methodologies involved in the functionality of *TransLib*, including the graph structure awareness to enable expertise engagement, methods for mining and exploring design spaces, management of the template library, and the realization of hardware operator generation.

4.1 Graph Structure Awareness

In the past extensive work conducted on accelerator design, a great deal of expertise and design techniques have been proposed [10, 17, 19], including structure-specific acceleration, operator and layer fusion, *etc.* However, no existing approach enables the easy incorporation of these techniques into an automated design process. Considering that these optimizations are primarily performed for specific graph structures in the network, *TransLib* applies an innovative algorithm for graph matching to attain expertise engagement. This approach can be illustrated as follows.

Stepwise-Contraction Analysis. To describe the target graph structure corresponding to specific optimization, an efficient description format is required. An operator in target network structure is selected as a keypoint in *TransLib* to simplify graph analysis and subsequent matching process. In addition to the regular set of vertices and edges, secondary information related to this keypoint can also be obtained. The specific algorithm of stepwise-contraction analysis is shown in Fig. 2. First, all vertices in the vicinity of the original keypoint are labeled with a distance of 1, and associated edges are removed. Next, these marked vertices are fused to create

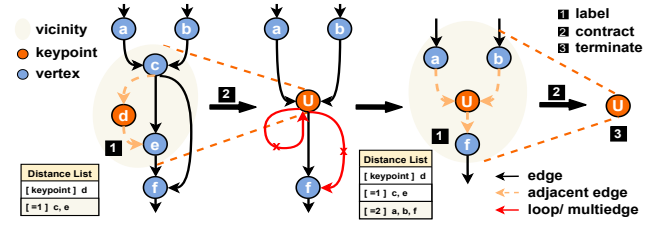


Figure 2: Graph algorithm for stepwise-contraction analysis with (1) label, (2) contract and (3) terminate.

a new union keypoint, and any resulting duplicate edges are eliminated. Then, the next round of contraction proceeds and relevant vertices are labeled with a distance of 2. This contraction process is repeated until there is only one single union keypoint in the target network graph. So far, all vertices of operators in the target structure are traversed and the step distances relative to the original keypoint is obtained.

Keypoint-Centric Matching. Transformer networks have a complex network structure, making it completely unfeasible to use standard adjacency matrix representations directly. Moreover, unlike classical graph mining tasks, specific optimization in transformer networks requires not only a structurally correct graph but also internal operator characteristics consistent with the target. To address these concerns, *TransLib* employs a recursive graph algorithm to perform keypoint-centric matching. The process occurs in two primary phases as follows.

Phase 1: Subdomain Extraction. After conducting the preliminary analysis, the maximum step distance has been established, enabling the extraction of desired subdomains. First, the eligible operators within the provided transformer network model are identified by analyzing the internal keypoint information in the templates. Subsequently, the vicinity domain within the network is expanded at the

maximum step distance for each potential keypoint, facilitating the extraction of subdomains. Consequently, multiple smaller subdomains, which potentially encompass the corresponding optimized regions, are identified for further processing.

Phase 2: Traversal Comparison. The recursive algorithm that implements the traversal comparison is displayed in Algorithm 1. Traversal starts with comparison between the original keypoints (line 3). If the connectivity and internal properties of the keypoints are the same, they will be marked as the same to avoid comparison again in subsequent traversal (line 2) and initiate follow-up comparison. Then, a straightforward match is made within the vicinity of these two original keypoints, and the traversal function is called again for each pair of matches as a new centroid (line 7~line 10). Therefore, a recursive comparison procedure with radial steps centered on keypoints is formed. This procedure has two exits: one for the border in the target graph template which means an exact match (line 5); and one for any other arbitrary mismatch (line 13).

Algorithm 1: Traversal Comparison

Input: potential subdomains s_p and its keypoint kp_p , target template graph g_t and standardized keypoint kp_t .
Output: Matching or not (True or False)

```

1 Traversal_Compare( $s_p, kp_p, g_t, kp_t$ ):
  // avoiding a dead loop
2   if marked as the same before then return True ;
3   if Check_Equal( $kp_p, kp_t$ ) is True then
4     mark  $kp_p$  is the same as  $kp_t$ ;
5     if  $kp_t$  is boundary op then return True ;
6     else
7        $V_t \leftarrow \text{Vicinity}(kp_t)$  ;
8        $V_p \leftarrow \text{Vicinity}(kp_p)$  ;
9       for  $v_p, v_t$  in  $V_p, V_t$  do
10        return Traversal_Compare( $s_p, v_p, g_t, v_t$ ) ;
11    end
12  end
13  else return False ;

```

At this point, a graph matching algorithm based on the target template is completed. Instead of conventional representation of the adjacency matrix, this keypoint-centric matching approach is more suitable for large and sparse network structures of transformer. Applying this algorithm, *TransLib* is able to efficiently detect the optimized network structure and achieve expertise engagement.

4.2 Design Space Mining and Exploration

The expansive design space of transformer accelerators is widely recognized in the field. However, when it comes to automated tools focused on network modelproto, effectively navigating the design space poses a significant challenge. Addressing this challenge, *TransLib* initially mines the design space from various dimensions, and then completes the exploration of the exposed design space.

4.2.1 Space Dimension Mining. *TransLib* employs a three-pronged approach to mining the design space and providing developers with accessible solutions for flexible generation of transformer operators. Details of these space dimensions are as follows.

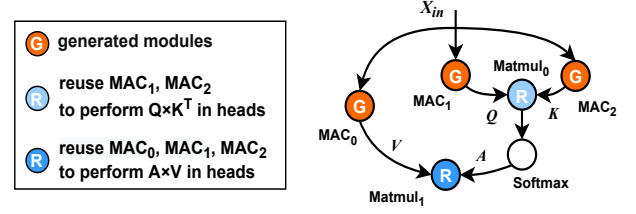


Figure 3: MAC-Matmul repeatability in multi-head attention.

Repeatability of network structure. Network structure of transformer has multiple levels of repetition, including decode blocks, operator groups and fine-grained operators. *TransLib* leverages parser and optimizer in the frontend to discover the above repeatability and then pass some of it to the backend for building or evaluation and expose the rest to the developer.

Specifically, *TransLib* evaluation takes into consideration the repeatability both inside and outside the transformer block. Inside the block, operators with similar properties and strictly sequential relationships can be effectively optimized and reused. For instance, Fig. 3 shows the computation flow in the multi-head attention stage. Computation time for Q , K , and V is nearly identical, enabling the subsequent matrix multiplication (Matmul) operation to reuse the large-scale multi-accumulate (MAC) modules with better performance. On the block level, developer can decide the number of blocks to be generated. On the one hand, designs with multiple blocks minimize on-chip and off-chip data exchange through DDR memory access. On the other hand, designs with fewer blocks facilitate the reuse of modules, as previously mentioned.

Flexibility of operator implementation. There are two types of design space for building operator modules as well. On the one hand, corresponding to a single operator, there can be diverse generative methods or specific optimizations in the template library. On the other hand, for the same generative algorithm for the same operator, configurations with different parameters also give different results. Considering that the transformer networks are generally of large scales, *TransLib* defaults to the most resource-friendly method, but also provides interfaces for modification.

Dataflow Scheduling. Although a simpler static scheduling is used due to the stable dataflow of the network accelerator, design space still exists due to the aforementioned repeatability. In conjunction with varying degrees of unit reuse and buffer insertion, different forms of dataflow scheduling can be generated.

4.2.2 Exploration Algorithm. The variety of operator combinations makes fully mining of design space completely unfeasible. Practically, operator implementation is constrained to resource and performance specifications, leading to specific implementation strategies for different application scenarios. Therefore, our study focuses on the operator subspaces from these strategies, summarized as resource-priority pruning (Prune-R) and performance-priority pruning (Prune-P), to achieve more efficient mining.

While Prune-R strategy can simply take the smallest templates as the subspace, Prune-P strategy needs to consider the performance tradeoffs under specific resource constraint, making it difficult to determine the subspace. To tackle the problem, *TransLib* applies several algorithms to target the above subspaces and carry out pruned design space mining subsequently.

Implementation Modeling. Modeling of implementation results is necessary for the application of algorithms. The total latency and computational resource utilization can be estimated respectively by the cumulative maximum latency and computational resource utilization, which can be summarized as follows:

$$L = \max_{path \in G} \left[\sum_{i \in path} (L_i \times \frac{N_i}{Y_i}) \right], \quad R = \sum_{i \in G} (R_i \times Y_i),$$

where L and R are the estimated latency and computational resources respectively, and G represents the graph of dataflow. Regarding reuse of repeated operator blocks, N_i is the total number of required blocks, and Y_i indicates the number of actually instantiated blocks, ranging from 1 to N_i . Therefore, $\frac{N_i}{Y_i}$ represents the iterations required to complete the computation.

In the above modeling, the latency L_i of a single operator i is calculated as follows:

$$L_i = cycles \times \frac{1}{f_{max}} \times 0.001,$$

where $cycles$ represents the clock cycles required for the operation, and f_{max} represents the maximum working frequency on target platform, which is determined as the minimum of the maximum frequencies of each operator to provide a realistic overall estimation.

Considering reuse of memory units, the total memory resource utilization can be estimated by the instant maximum memory requirement of the computational graph, which is the maximum requirement for serial paths and sum of requirements for parallel paths. For transformer blocks, the computational graph G is first split into serial subgraphs, and each subgraph is made up of single or parallel paths, marked as G_p . Then the memory estimation can be summarized as follows:

$$M = \max_{G_p \in G} \left(\sum_{path \in G_p} M_{path} \right),$$

where M is the estimated memory resources. As M_{path} is the memory cost for a single path, M_{path} can be similarly calculated using the maximum function.

Genetic Algorithm. In the Prune-P process, *TransLib* applies genetic algorithm (GA) to identify the optimal operator templates that leads to the best performance. For algorithm setup, the design space mining is regarded as a mixed integer programming (MIP) problem, where the total resources of target FPGA are set as the constraint, the number of heads and the attributes of each generated operator, i.e., the operator size, DSP-based multiplier or LUT-based multiplier, are set as variables and genes.

The key component of the algorithm is the fitness function, with the objective of minimizing latency to enhance performance. The fitness function $Fit(L)$ for a given latency L (ms) is expressed as: $Fit(L) = 100L^{-2}$, where L refers to the latency of a single transformer block, and L^{-2} is chosen to control the convergence speed of population fitness so that the evolution process can be observed obviously while proceeding efficiently.

With the fitness function and structure, genes undergo crossover and mutation to produce offspring, of which the elite offspring are selected as new parents according to the fitness function. Through several iterations, the best candidate from the population is selected

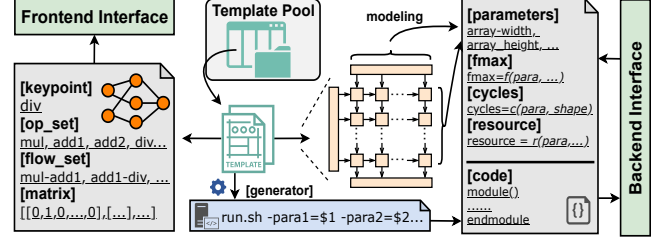


Figure 4: Management of template pool and bidirectional interfaces in *TransLib* library.

as the predicted result. Parameters from the result are then fitted into the template library to derive an operator combination.

4.3 Library Management

The library is the crucial infrastructure that facilitates the functionality and extensibility of *TransLib*. To attain these objectives, the library is organized into the following two parts, as shown in Fig. 4.

Template pool. Various optimization templates for specific network structures or operators are stored in the template pool of *TransLib*. According to the forms of hardware templates, they are divided into two categories: *generator* mode and *code* mode. The former contains different existing high-performance operator generators, while the latter is made up directly of hardware code templates such as HDL code. All the templates and necessary parameters are listed in the library configuration file, which provides available generation methods for the backend. The template pool and library configuration are extensible, allowing for customized tools and configuration from users.

Through the template library configuration, the templates for these two modes are taken and used differently in the builder for subsequent operator generation.

Bidirectional interfaces. In order to ensure proper generation functioning and at the same time achieve extensibility, *TransLib* adopts a bidirectional interface management. On the one hand, a formatted description file of the target optimized structure is provided for each template. These description files are combined into the library's information file, which is then exported to frontend tools via interfaces with standardized formats, including template keypoints, inner operators and dataflow. On the other hand, different generative methods or templates require different configuration parameters for building. Another inbound interface is created so that backend tools can access the correct target and provide the necessary attributes.

4.4 Operator Set Generation

After performing structure-aware optimization and design space mining, the operator generation is able to proceed through the *TransLib* backend, as shown in Fig. 5.

Specifically, the builder generates parameter-specified operator sets with the templates from the library according to the configuration information of the operators in the network, including operator type, data shape, template parameters, etc. Based on the optimized network topological structure, port alignment is then carried out for the inputs and outputs, so that generated operator sets can support subsequent module interconnection with efficiency.

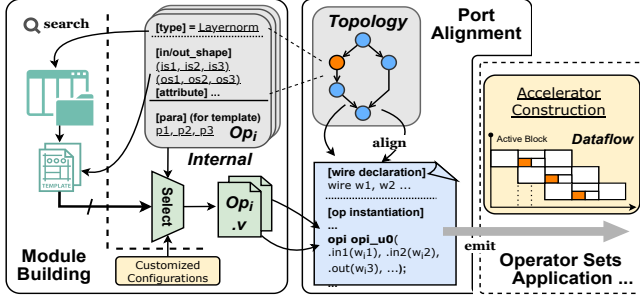


Figure 5: Module building and port alignment of operator sets in *TransLib* backend.

In consistence with the template library, the builder generates operator sets in different generation modes. To be specific, the builder first reads the network information extracted from the frontend, including operator configurations and network topology, *etc.* Then for each type of operator, it searches the template library configuration for available generation modes and methods, displays available choices for the user, and generates modified operator sets with the method user specifies.

For the *code* mode, the builder directly uses the operator modules in the template library and modifies the parameters listed in the library configuration. The builder script uses regular expressions to match both parameters and complete the modification. For the *generator* mode, the builder will also encapsulate the required parameters into executable command files that automatically connect to and call the appropriate generators.

The operator sets generated possess wide-ranging applications in accelerator construction. Initially, port alignment facilitates the straightforward connection and extension of interfaces. Additionally, prior exploration of the design space enhances flexible dataflow scheduling, thereby enabling the automated development of transformer accelerators with extensive architectures.

5 EVALUATION AND CASE STUDIES

5.1 Experimental Setup

To evaluate proposed *TransLib* framework, we conduct comprehensive experiments and ablation case studies in this section. The template pool of *TransLib* is built on multiple types of realizations, including both existing high-performance templates and handcraft operators, like non-linear operators achieved by NN-LUT [26], matrix multiplication achieved by systolic arrays and adder tree, *etc.* The quantization methods used in the experiments are referred to the implementation in IBERT[14] to INT8, unless otherwise noted.

The modelproto cases of transformer networks involved in the experiment are derived from Modelzoo of ONNX Community [6] or the transformer library of Hugging Face [23]. The reported performances and resources utilization are collected from the synthesis and implementation results reported by Vivado 2019.2.

In the following experiments outlined in this paper, we conduct generality evaluation to validate the capability of expertise engagement in *TransLib* frontend (in Section 5.2), followed by flexibility evaluation of operator generation in its backend (in Section 5.3). Lastly, we focus on case studies for different scenarios to demonstrate the specific performance of the predicted accelerator generated by *TransLib* (in Section 5.4).

Table 2: Generality evaluation for expertise engagement

Template	Graph Analysis			Target Matching		
	kpt	ty/nu	distance list	num	types	size_max
MAC	Mm	4/4	(0,1,1,2)	72/48	3/4	[seq, 3072, 768]
Madd	Add	1/4	–	24	1	[seq, 768]
Matmul	Mm	1/4	–	24	2	[seq, seq, 64]
Gelu	Erf	4/7	(0, 1, 1, 2, 2, 3, 3)	12	1	[seq, 3072]
LayerNorm	Pow	7/12	(0, 1, 1, 2, 2, 3, 3, 3, 4, 5, 6, 7)	24	1	[seq, 768]
Softmax	–	1/1	–	12	1	[seq, seq]
Reshape	Iden	4/4	(0,1,1,2)	48	3	[seq, 768]

kpt: Keypoint; ty/nu: Number of types/ of operators in template;

5.2 Generality

To verify the generality of graph awareness in our work, we first integrate various optimization templates in the template pool in proposed *TransLib* and conduct aforementioned stepwise-contraction analysis and keypoint-centric matching to achieve expertise engagement in some typical transformer networks, including Bert, Roberta, ViT, DeiT, and GPT-2.

For different optimization templates, the results of target graph analysis and matching detection in each network are shown in Table 2. It can be seen that graph analysis over the target structures of different templates is successfully completed. All levels of properties in the target structures are extracted correctly, including the basic types and numbers of internal operators and also the topological location relative to the keypoint. Note that there are a variety of options about the keypoint operator in each template, while this only influences the relative analysis description and not the correctness of actual target structures or following matching. Moreover, *TransLib* identifies and locates the optimization domains in each transformer network for different templates. Specifically, for templates of different sizes, the total number and types are extracted for utilization evaluation, while the maximum size is used to evaluate the performance bottleneck in the design. The success in the matching detection of the target structures indicates the realization of expertise engagement and proves its generality.

Fig. 6 illustrates the number of operators and types in modelproto for different transformer networks before and after the process of *TransLib* frontend. In the original networks, *TransLib* finds operators directly equivalent to existing templates, while structurally optimizing numerous fine-grained operators and removing redundant operators for template mapping. Evidently, the operator complexity of different networks decreases by 44.2% to 91.8%. This

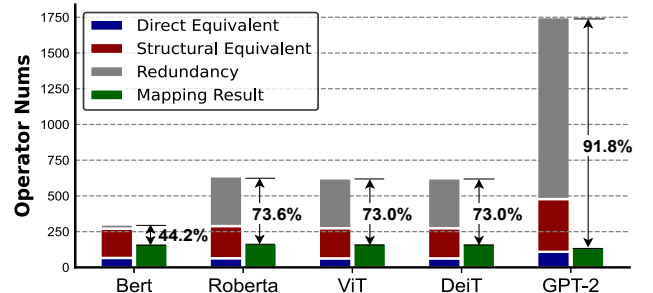


Figure 6: Operator complexity of transformer networks before and after graph awareness in *TransLib*.

Table 3: Dimension results of design space mining for different networks and implementation strategies.

Design Space	Rep	Op Imp			DF	Total
		Prune-R	Prune-P	Total		
B/Rb	6	2.07×10^6	5.31×10^8	1.36×10^{11}	6	4.90×10^{12}
G	6	1.73×10^5	2.76×10^6	7.09×10^8	6	2.55×10^{10}
V/De	6	6.91×10^5	1.77×10^8	4.53×10^{10}	6	1.63×10^{12}

Rep: Repeatability; Op Imp: Operator Implementation; DF: Dataflow.
B: Bert; Rb: Roberta; V: ViT; De: DeiT; G: GPT-2.

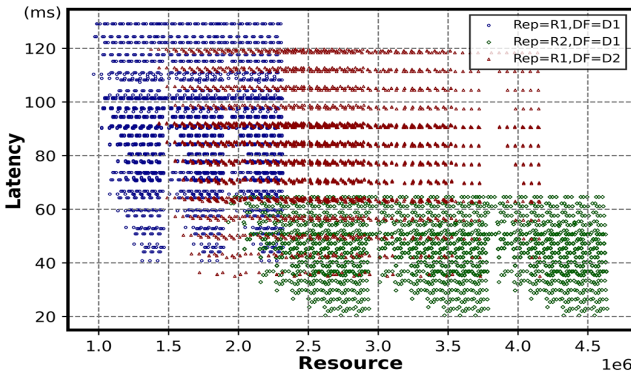
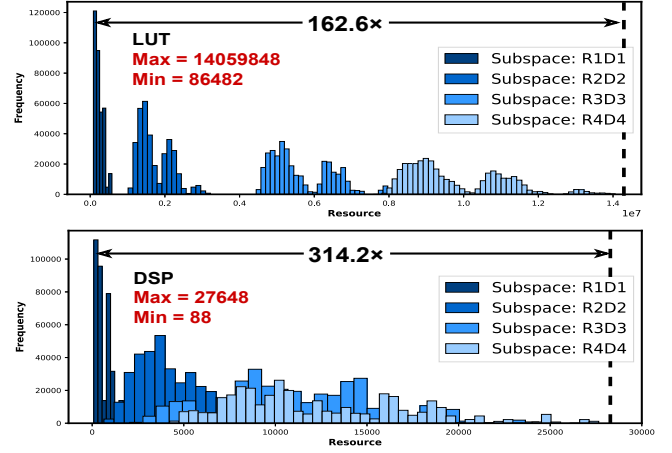
showcases the capability of *TransLib* to deconstruct and optimize various transformer networks with a uniform template library.

5.3 Flexibility

Design space mining in *TransLib* unveils significant flexibility in accelerator generation. Table 3 summaries partial descriptive design space dimensions mined in *TransLib* in accelerator generation for different transformer networks. For these diverse transformer networks, *TransLib* exploits the design space in three dimensions, resulting in excellent overall flexibility with 10^{10} to 10^{12} different generations. For repeatability and dataflow, adaptive dimensions are determined by the specific structure of each network. Furthermore, although the template library is the same in *TransLib*, differences in the data shape also allow in diverse flexibility. Additionally, two pruning strategies for different referencing scenarios both bring down the complexity for over 4092 $\times$ and 256 $\times$, which substantially increases mining efficiency.

To visually represent the design space, several subspaces are chosen to display and compare. For subspace representation, we consider the aforementioned network repeatability (Rep) and dataflow scheduling (DF) in design space mining, and mark the subspace as R_iD_j . Here higher i and j represents higher block repeatability and parallelism of dataflow respectively. Fig. 7 displays the resource utilization and latency results in three specific subspaces. Within each subspace, numerous operators are implemented in different strategies, resulting in a detailed exploration of the space. Comparing each of the two subspaces reveals that various strategies for repeatability and dataflow impact the tradeoffs on a larger scale.

The distribution of hardware resources of each generation in different subspaces is counted to obtain Fig. 8. It is observed that

**Figure 7: Visualization of resources utilization and latency results in three specific subspaces with different strategies.****Figure 8: Distribution of resources utilization in different subspaces of the design space, for LUT(above) and DSP(below).**

the resource consumption of the accelerator generation in *TransLib* spans up to 162.6 $\times$ in LUT and 314.2 $\times$ in DSP for the same network. Note that there are numerous subspaces situated between the two boundary subspaces, sufficient to cover almost every region of the period. The broad range of available hardware resources demonstrates the versatility of the operator templates of *TransLib* and its potential for use on platforms with varying limitations.

Based on above analysis of different subspaces, we further conduct GA with restrictions from different platforms to derive smaller and preciser subspaces with better latency. Fig. 9 shows fitness improvement over generations in GA. With the initial population, GA is able to reach optimal fitness within 12 generations and the execution time 3.79s on average, while exhibiting obvious stages of stagnation and fitness evolution in the process. The derived parameters from GA results are applied to generate optimal cases of operator sets for each of the platform, which are later used for implementation and performance evaluation. Details of these cases will be discussed in Section 5.4.

5.4 Case Studies

We utilize FPGAs of different specifications as hardware platforms in carrying out the operator deployment analysis of *TransLib*. Specifically, to evaluate the overall implementation results of the generative operators, we first conduct implementation of target models

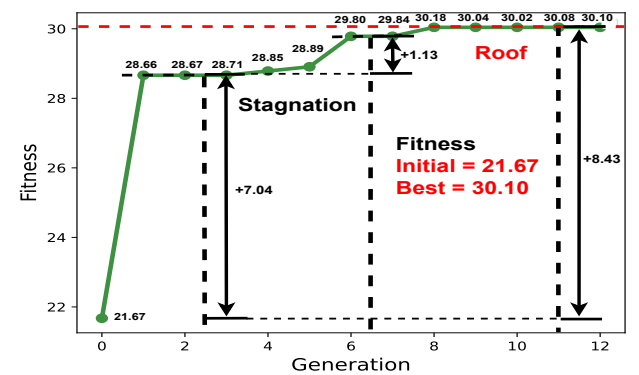
**Figure 9: Fitness improvement over generations during GA process in the design space exploration of *TransLib*.**

Table 4: Implementation performance of different transformer operator sets generated by *TransLib*.

Network	Input Shape Data Type	Platform	Utilization				Latency		
			LUT	FF	DSP	BRAM	CPU	GPU	Ours
Bert-base	[1,128]	XC7K325T	178176 / 87.43%	348959 / 85.61%	680 / 80.95%	319 / 71.69%	48.05ms	7.45ms	957.36ms
	INT8	Alveo U280	918117 / 70.43%	1422904 / 54.57%	8400 / 93.09%	624 / 30.95%			12.65ms
ViT-base	[1, 3, 224, 224]	XC7K325T	183841 / 90.21%	337387 / 82.77%	712 / 84.76%	319 / 71.69%	57.00ms	8.83ms	1547.55ms
	INT8	Alveo U280	873907 / 67.03%	1374458 / 52.71%	8464 / 93.38%	624 / 30.95%			23.96ms
GPT-2	[1,128]	XC7K325T	182774 / 89.68%	353596 / 86.75%	616 / 73.33%	319 / 71.69%	141.78ms	71.00ms	4018.42ms
	INT8	Alveo U280	860471 / 66.00%	1393604 / 53.45%	8368 / 92.73%	624 / 30.95%			63.61ms

CPU: Intel Core i7-12700H; GPU: NVIDIA RTX 3090.

on the respective FPGA platforms. Then we compare our inference performance with reference to different platforms, including CPU, GPU and existing works of FPGA implementation tools.

Table 4 presents the estimated results of operator implementations in *TransLib* under different FPGAs of the edge device (*Kintex-7 325T*) at 50MHz and the data center board (*Alveo U280*) at 150MHz. The operators generated by *TransLib* for different networks can be seen to flexibly use the computational resources, with a high utilization rate of LUT from 87.43% to 90.21% and FF from 82.77% to 86.75% for the edge device, and a moderate utilization rate for the data center FPGA, where LUT utilization falls to 66.00% to 70.43% and DSP utilization rises to 92.73% to 93.38%, marking higher utilization of computational resources instead of logical resources on large devices. The flexibility in implementation is highly attributed to aforementioned design space. Considering that route design fall severely when over-utilized, the maximum utilization rate has been restricted to around 90% for the edge device.

Table 5 presents the performance comparison on different platforms. The experiments are carried out on the Bert network under the sequence of 128, the most commonly used sequence length in Bert. It shows that *TransLib* achieves 3.80-11.47 $\times$ speedup compared to common CPU platforms and 2.74-4.43 $\times$ speedup compared to previous works of ASIC implementation without DSP and BRAM. Moreover, *TransLib* achieves similar results compared to manual tools of FPGA implementation, indicating competitive performance of our automated operator sets with reference to existing manual works of transformer accelerators. Note that as *TransLib* is an entirely automated framework for generating operator sets without any manual optimization, it can be regarded as competitive in the automation of accelerator generation.

In terms of inference performance, we only estimate the total computational latency, excluding minor latency caused by control logic. It shows that operators generated for the data center FPGA achieve competitive results compared to the general platform, while deployment at the edge device proves flexibility of *TransLib*, even when performance is limited by on-chip resources. To be mentioned, it is foreseeable that operators generated by *TransLib* can attain increasingly better performance with future expansion and optimization in the template library.

6 CONCLUSION

In this paper, we propose *TransLib*, an extensible library-based framework for auto-generation of transformer operators. Specifically, we first attain graph structure awareness with innovative

Table 5: Comparisons of inference performances of BERT-128 on different platforms or previous work.

Platform/Work	Type	DSP	BRAM	Latency	Speedup
				(ms)	ratio
Intel Core i7-8700	CPU	\	\	145.06	1.00
Intel Xeon Gold 5218R	CPU	\	\	50.00	2.90
Intel Core i7-12700H	CPU	\	\	48.05	3.02
NVIDIA K80	GPU	\	\	27.84	5.21
NVIDIA RTX K5000	GPU	\	\	10.61	13.67
NVIDIA RTX 3090	GPU	\	\	7.45	19.47
A3 [10]	ASIC	\	\	56.00	2.59
ELSA [11]	ASIC	\	\	34.70	4.18
HAQ [19]	FPGA	4272	1358	23.79	6.10
NPE [13]	FPGA	2020	369	13.96	10.39
FTRANS [17]	FPGA	6531	\	10.61	13.67
FET-OPU [2]	FPGA	4864	1357	6.82	21.27
TransLib	FPGA	8400	624	12.65	11.47

graph algorithms for expertise engagement in the frontend. Then, the backend accomplishes design space mining and operator generation for different platforms or scenarios. Moreover, a pluggable library is built with concise and bidirectional interfaces to make *TransLib* extensible for customizable designs. Experiments demonstrate that *TransLib* has the capability of generating multiple transformer operators generically and flexibly, while balancing resource utilization and accelerator performance on different hardware platforms. Therefore, *TransLib* can support the continuously evolving transformer networks by expanding its template library and incorporating expertise, swiftly offering competitive performance in the acceleration of emerging transformer algorithms.

ACKNOWLEDGMENTS

The authors thank the anonymous reviewers for their valuable suggestions. This work was financially supported in part by National Key Research and Development Program of China under Grant 2021YFA1003602, in part by Shanghai Pujiang Program under Grant 22PJJD003. The computations in this research were performed using the CFFF platform of Fudan University.

REFERENCES

- [1] Nicolas Bohm Agostini et al. 2022. An MLIR-Based Compiler Flow for System-Level Design and Hardware Acceleration. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design* (San Diego, California) (ICCAD '22). Association for Computing Machinery, New York, NY, USA, Article 6, 9 pages. <https://doi.org/10.1145/3508352.3549424>
- [2] Yueyin Bai et al. 2023. FET-OPU: A Flexible and Efficient FPGA-Based Overlay Processor for Transformer Networks. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9. <https://doi.org/10.1109/ICCAD57390.2023.10323752>
- [3] Yueyin Bai et al. 2023. LTrans-OPU: A Low-Latency FPGA-Based Overlay Processor for Transformer Networks. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. 283–287. <https://doi.org/10.1109/FPL60245.2023.00048>
- [4] Tianqi Chen et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.
- [5] CIRCT Community. 2023. CIRCT: Circt IR Compilers and Tools. <https://github.com/llvm/circt>.
- [6] ONNX Community. 2023. Open Neural Network Exchange. <https://github.com/onnx>.
- [7] Jacob Devlin et al. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [8] Alexey Dosovitskiy et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [9] Luciano Floridi et al. 2020. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines* 30 (2020), 681–694.
- [10] Tae Jun Ham et al. 2020. A3: Accelerating Attention Mechanisms in Neural Networks with Approximation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 328–341. <https://doi.org/10.1109/HPCA47549.2020.00035>
- [11] Tae Jun Ham et al. 2021. ELSA: Hardware-Software Co-design for Efficient, Lightweight Self-Attention Mechanism in Neural Networks. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 692–705. <https://doi.org/10.1109/ISCA52012.2021.00060>
- [12] Liancheng Jia et al. 2021. TensorLib: A Spatial Accelerator Generation Framework for Tensor Algebra. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 865–870. <https://doi.org/10.1109/DAC18074.2021.9586329>
- [13] Hamza Khan et al. 2021. NPE: An FPGA-Based Overlay Processor for Natural Language Processing. In *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Virtual Event, USA) (FPGA '21). Association for Computing Machinery, New York, NY, USA, 227. <https://doi.org/10.1145/3431920.3439477>
- [14] Sehoon Kim et al. 2021. I-bert: Integer-only bert quantization. In *International conference on machine learning*. PMLR, 5506–5518.
- [15] Chris Lattner et al. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004*. IEEE, 75–86.
- [16] Chris Lattner et al. 2021. MLIR: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2–14.
- [17] Bingbing Li et al. 2020. FTRANS: Energy-Efficient Acceleration of Transformers Using FPGA. In *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design* (Boston, Massachusetts) (ISLPED '20). Association for Computing Machinery, New York, NY, USA, 175–180. <https://doi.org/10.1145/3370748.3406567>
- [18] Yang Liu et al. 2023. DIF-LUT: A Simple Yet Scalable Approximation for Non-Linear Activation Function on FPGA. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. 322–326. <https://doi.org/10.1109/FPL60245.2023.00055>
- [19] Zejian Liu et al. 2021. Hardware Acceleration of Fully Quantized BERT for Efficient Natural Language Processing. In *2021 Design, Automation Test in Europe Conference Exhibition (DATE)*. 513–516. <https://doi.org/10.23919/DATE51398.2021.9474043>
- [20] Patrick Plagwitz et al. 2022. TRAC: Compilation-Based Design of Transformer Accelerators for FPGAs. In *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. 17–23. <https://doi.org/10.1109/FPL57034.2022.00015>
- [21] Yaman Umuroglu et al. 2017. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Monterey, California, USA) (FPGA '17). Association for Computing Machinery, New York, NY, USA, 65–74. <https://doi.org/10.1145/3020078.3021744>
- [22] Ashish Vaswani et al. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [23] Thomas Wolf et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45. <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [24] Pengfei Xu et al. 2020. AutoDNNchip: An Automated DNN Chip Predictor and Builder for Both FPGAs and ASICs. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Seaside, CA, USA) (FPGA '20). Association for Computing Machinery, New York, NY, USA, 40–50. <https://doi.org/10.1145/3373087.3375306>
- [25] Hanchen Ye et al. 2022. ScaleHLS: A Scalable High-Level Synthesis Framework with Multi-Level Transformations and Optimizations: Invited. In *Proceedings of the 59th ACM/IEEE Design Automation Conference* (San Francisco, California) (DAC '22). Association for Computing Machinery, New York, NY, USA, 1355–1358. <https://doi.org/10.1145/3489517.3530631>
- [26] Joonsang Yu et al. 2022. NN-LUT: Neural Approximation of Non-Linear Operations for Efficient Transformer Inference (DAC '22). Association for Computing Machinery, New York, NY, USA, 577–582. <https://doi.org/10.1145/3489517.3530505>